

Class 6: R functions

Barry (PID: 911)

Table of contents

Our first (silly) function	1
A second function	2
A protein generating function	3

All functions in R have at least 3 things:

- A **name**, we pick this and use it to call our function,
- Input **arguments** (there can be multiple)
- The **body** lines of R code that do the work

Our first (silly) function

Write a function to add some numbers

```
add <- function(x, y=1) {  
  x + y  
}
```

Now we can call this function:

```
add(c(10, 10), 100)
```

```
[1] 110 110
```

```
add(10, 100)
```

```
[1] 110
```

A second function

Write a function to generate random nucleotide sequences of a user specified length:

The `sample()` function can be helpful here.

```
v <- sample(c("A","C","G","T"), size=50, replace = TRUE)
v
```

```
[1] "G" "G" "A" "G" "C" "A" "C" "A" "C" "T" "A" "G" "A" "T" "G" "C" "T" "T" "G"
[20] "C" "T" "T" "A" "C" "C" "A" "A" "G" "C" "A" "A" "G" "C" "G" "A" "A" "T" "A"
[39] "C" "A" "T" "C" "G" "C" "G" "C" "C" "C" "A" "A"
```

I want the a 1 element long character vector that looks like this “CACAGC” not “C” “A” “C”
“A” “G” “C”

```
v <- sample(c("A","C","G","T"), size=50, replace = TRUE)
paste(v, collapse = "")
```

```
[1] "CTACGTCTGTGTAATTAGATATTTAGCTGTTTTGTGTCCTGTCATAATAT"
```

Turn this into my first wee function

```
generate_dna <- function(size = 50) {
  v <- sample(c("A", "C", "G", "T"), size = size, replace = TRUE)
  return( paste(v, collapse = "") )
}
```

Test it:

```
generate_dna(60)
```

```
[1] "ACGGCTCCAAATCAGGAGAATTGCTTTGATCCACACCGGCTGTTAGCCGTTTTGTCTGAT"
```

```
fasta <- TRUE
if(fasta) {
  cat("HELLO You!")
} else {
  cat("No you dont!")
}
```

HELLO You!

Add the ability to return a multi-element vector or a single element fasta like vector.

```
generate_fasta <- function(size = 50, fasta=TRUE) {  
  v <- sample(c("A", "C", "G", "T"), size = size, replace = TRUE)  
  s <- paste(v, collapse = "")  
  
  if(fasta) {  
    return(s)  
  } else {  
    return(v)  
  }  
}
```

```
generate_fasta(10)
```

```
[1] "AGCCTGGATC"
```

```
generate_fasta(10, fasta=FALSE)
```

```
[1] "A" "T" "T" "A" "G" "C" "T" "G" "T" "T"
```

A protein generating function

```
generate_protein <- function(size = 50, fasta = TRUE) {  
  aa <- c("A", "R", "N", "D", "C", "Q", "E", "G", "H", "I",  
          "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")  
  v <- sample(aa, size = size, replace = TRUE)  
  s <- paste(v, collapse = "")  
  
  if (fasta) {  
    return(s)  
  } else {  
    return(v)  
  }  
}
```

```
generate_protein(6)
```

```
[1] "PIAEEV"
```

Use our new `generate_protein()` function to make random prteon sequences of length 6 to 12 (i.e. one length 6, one length 7, etc. up to lenght 12).

One way to do this is “brute force”

```
generate_protein(6)
```

```
[1] "WGWYPK"
```

```
generate_protein(7)
```

```
[1] "VNLMFCS"
```

```
generate_protein(8)
```

```
[1] "NVVCLSAQ"
```

```
generate_protein(9)
```

```
[1] "YPWPISNWL"
```

A second way is to use a `for()` loop:

```
lengths <- 6:12  
lengths
```

```
[1]  6  7  8  9 10 11 12
```

```
for(i in lengths) {  
  cat(">", i, "\n", sep="")  
  aa <- generate_protein(i)  
  cat(aa)  
  cat("\n")  
}
```

```
>6
LHCAYQ
>7
TTSMFFC
>8
ITGLHLKT
>9
PAYLQSVGV
>10
FAVEDHRTSS
>11
KEVDGWQAIRI
>12
PVTSYFKRDEPY
```

A third, and better, way to solve this is to use the `apply()` family of functions, specifically the `sapply()` function in this case.

```
sapply(6:12, generate_protein)
```

```
[1] "CFYPMP"      "NKLGDAQ"      "SFTQPAGS"      "TSVLRWTEA"      "TKYTDKQVFS"
[6] "VACVVFDLYSL" "AQLHQIRPCQCT"
```